

Topics to cover

1. SDLC
2. SDLC models
 - a. Waterfall
 - b. Agile
3. Software testing
 - a. Functional and Non functional testing
 - b. Unit, System, Acceptance testing
 - c. Alpha and Beta testing
 - d. White and Black Box testing
 - e. Regression testing
 - f. Stress testing
 - g. Sandbox testing
4. Software requirements
 - a. Functional
 - b. Non functional
5. COCOMO model
6. Different diagrams
 - a. Use case diagram
 - b. State diagram
 - c. Class diagram
 - d. Sequence diagram
 - e. PERT chart
 - f. Gantt chart

Software Development Life Cycle (SDLC)

Software Development Life Cycle (SDLC) is a systematic and organized process used to plan, design, develop, test, deploy, and maintain software systems in a planned way.

Steps or Phases of SDLC

1. Requirement Analysis

- Collect and analyze user needs and system requirements.
- Define what the software should do.

2. Planning

- Create a project plan (timeline, resources, cost, risks).
- Decide the development approach and feasibility.

3. System Design

- Design the system architecture, data flow, and interfaces.
- Prepare detailed design documents.

4. Development (Coding)

- Write the actual code based on the design.
- Develop modules and integrate them.

5. Testing

- Test the software for bugs, errors, and performance issues.
- Ensure it meets the requirements.

6. Deployment

- Release the software to users or the production environment.
- Can be a full or phased release.

7. Maintenance

- Fix bugs, update features, and improve performance.
- Provide ongoing support after deployment.

SDLC models

Waterfall model

The Waterfall Model is a linear and sequential software development life cycle (SDLC) model. In this model, the development process flows steadily downwards through specific phases—like a waterfall—where each phase must be 100% completed before the next one begins.

Phases	Description	Deliverable
Requirements	All possible requirements of the system are gathered and documented.	SRS (Software Requirement Specification)
System Design	Architects design the hardware, software architecture, and data models.	Design Document (Blueprints)
Implementation	The actual coding happens here. The system is built in small units.	Source Code
Integration & Testing	All units are combined and tested as a whole system for bugs.	Test Reports
Deployment	The product is released to the customer's environment.	Live Product
Maintenance	Fixing issues found by users and updating the system as needed.	Updates/Patches

Agile Model

Agile Software Development is an iterative and incremental approach to software engineering that focuses on delivering high-quality, working software through continuous collaboration, frequent releases, and an ability to respond rapidly to changing requirements.

Agile Software Development Life Cycle (SDLC) -

These are the macro-steps a project takes from an initial idea to the day the software is retired.

1. **Concept (Ideation):** The Product Owner identifies a business need, defines the project scope, and estimates time/cost.
2. **Inception (Initiation):** The team is formed, and initial requirements are gathered. This is where the Product Backlog (a prioritized list of features) is created.
3. **Iteration (Construction):** This is the heart of Agile. The team works through multiple "Sprints" to build the software piece by piece.
4. **Release (Deployment):** Once a set of features is ready, the software undergoes final testing (UAT), documentation is finalized, and the increment is pushed to production.
5. **Maintenance:** The team provides ongoing support, fixes bugs discovered by users, and keeps the system running smoothly.
6. **Retirement:** The software is eventually decommissioned or replaced by a new system.

Software testing

Functional Testing		Non-Functional Testing	
Verifies what the system does.		Verifies how well the system performs.	
Some examples - a. Unit Testing b. Integration Testing c. System Testing d. Smoke Testing e. Sanity Testing f. Regression Testing g. User Acceptance Testing (UAT) h. Alpha Testing i. Beta Testing j. Black Box Testing k. White Box Testing l. Interface Testing		Some examples - a. Performance Testing b. Load Testing c. Stress Testing d. Scalability Testing e. Usability Testing f. Reliability Testing g. Security Testing h. Compatibility Testing i. Accessibility Testing j. Installation Testing k. Recovery Testing l. Maintainability Testing	
Regression Testing	Testing existing functionality after changes to ensure no new defects have been introduced.	After adding a new payment method, testing all existing payment options to ensure they still work correctly.	
Stress Testing	Testing system behavior under extreme or beyond-capacity load to find breaking points.	Simulating 50,000 users logging into a banking app at once to see when it crashes.	
Sandbox Test	A Sandbox Test refers to security and system management techniques that includes executing software,		

Software requirements

Functional Requirements (FR) - Functional requirements define the specific behaviors, features, and functions that a system must provide. If these are not met, the system cannot perform its intended task.

Some examples -

- **Authentication & Authorization**
- **Business Logic/Rules**
- **Data Entry & Management**
- **Transaction Handling**
- **Reporting & Analytics**
- **External Integrations**
- **Error Handling**
- **Administrative Functions**

Non Functional Requirements (NFR) - Non-functional requirements (also known as Quality Attributes) describe the constraints and performance standards of the system. They ensure the system is reliable, fast, and easy to use.

Some examples -

- Performance:
 - **Response Time**
 - **Throughput**
 - Recovery time from failure
- Scalability
- Security
- Availability
- Usability
- Reliability
- Maintainability
- Portability
- Compliance/Legal

COCOMO model

The **Constructive Cost Model (COCOMO)** is a procedural software cost estimation model for software projects based on LOC (Line of code) developed by Barry Boehm in 1981.

Types of software projects - The COCOMO model can be applied to three types of software projects:

1. **Organic** project: An organic software project requires a small team, the problem is well understood, and the team members are experienced in developing similar projects.
2. **Semi-detached** project: A semi-attached project is medium-sized, with the development consisting of a mix of experienced and inexperienced staff, and the major characteristics of the project lie in between organic and embedded.
3. **Embedded** project: An embedded software project has a high level of complexity, requiring a large-sized experienced team, and is coupled with complex hardware.

Types of COCOMO models - The COCOMO model is divided into three types based on the accuracy quotient. Any of the three types can be adapted according to our requirements:

1. Basic model
2. Intermediate model and
3. Detailed model

Basic COCOMO model - The basic model is used for quick and rough cost calculations for the software. It calculates the effort, time, and number of people required to use a project's kLOC (kilo lines of code).

The formulae to calculate these entities are:

$$\text{Effort (E)} = a (\text{kLOC})^b$$

$$\text{Development time (T)} = c (E)^d$$

$$\text{People required} = E / T$$

The effort is measured in person-months and time in months. The constants a, b, c, and d may vary for each model type. The following are the constant values for the basic model:

Project Type	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Example - Suppose a project was estimated to be made in 400 kLOC. Let's calculate its effort, time, and the number of people required while considering the project is of organic type -

$$\begin{aligned}\text{Effort (E)} &= a (\text{kLOC})^b \text{ person/month} \\ &= 2.4 (400 \text{ kLOC})^{1.05} \text{ person/month} \\ &= 1295.31 \text{ person/month}\end{aligned}$$

$$\begin{aligned}\text{Development time (T)} &= c (E)^d \\ &= 2.5 (1295.31)^{0.38} \\ &= 30.07 \text{ month}\end{aligned}$$

$$\begin{aligned}\text{People required} &= E / T \\ &= 1295.31 / 30.07 \\ &= 43.07 \text{ person}\end{aligned}$$

Different diagrams

Categories of UML Diagrams	
A. Structural Diagrams - These show what the system is made of. They represent the static structure that does not change over time.	B. Behavioral Diagrams - These show how the system acts and how it responds to requests.
• Class Diagram • Object Diagram • Component Diagram	• Use Case Diagram • Sequence Diagram • Activity Diagram • State Machine Diagram

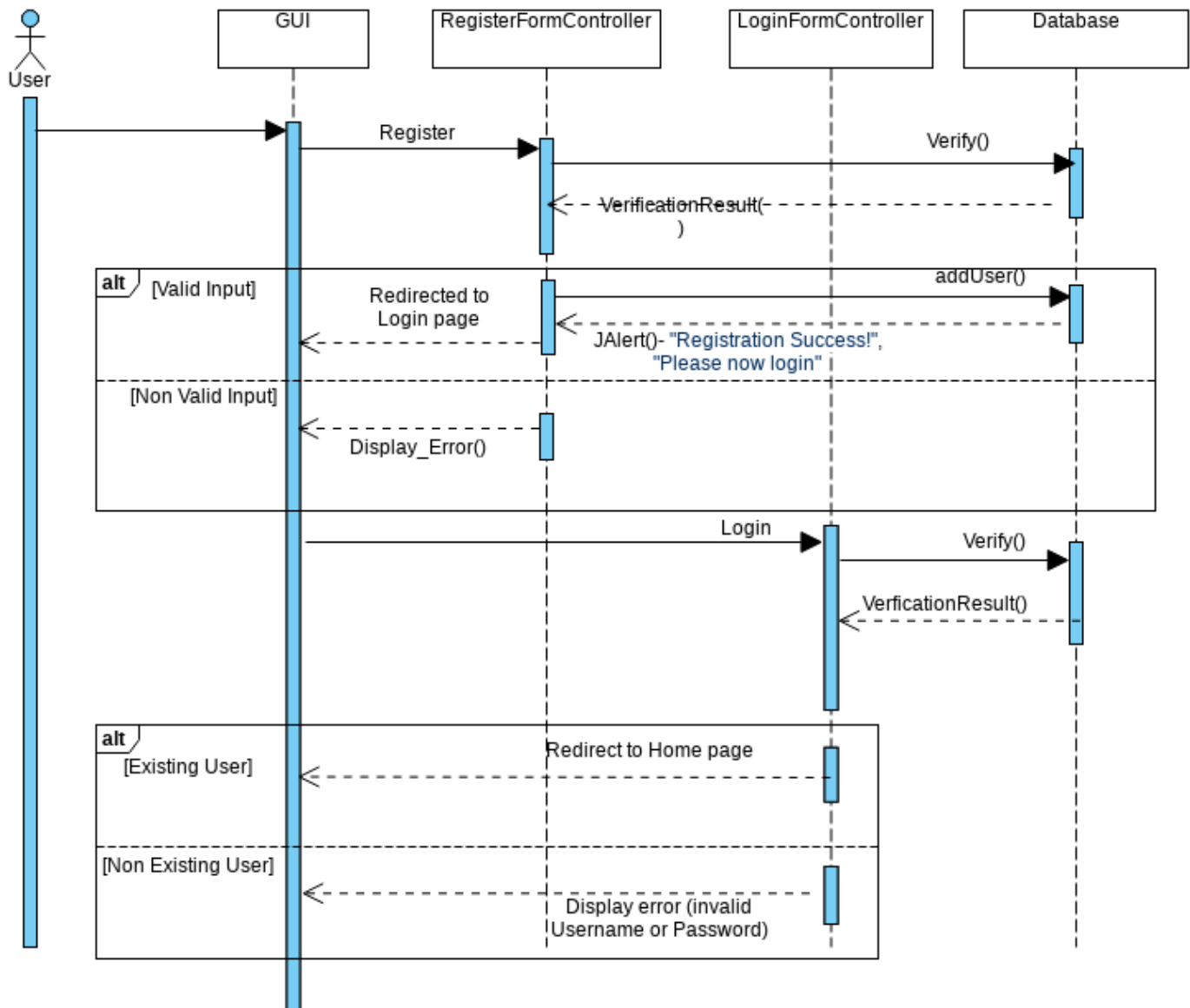
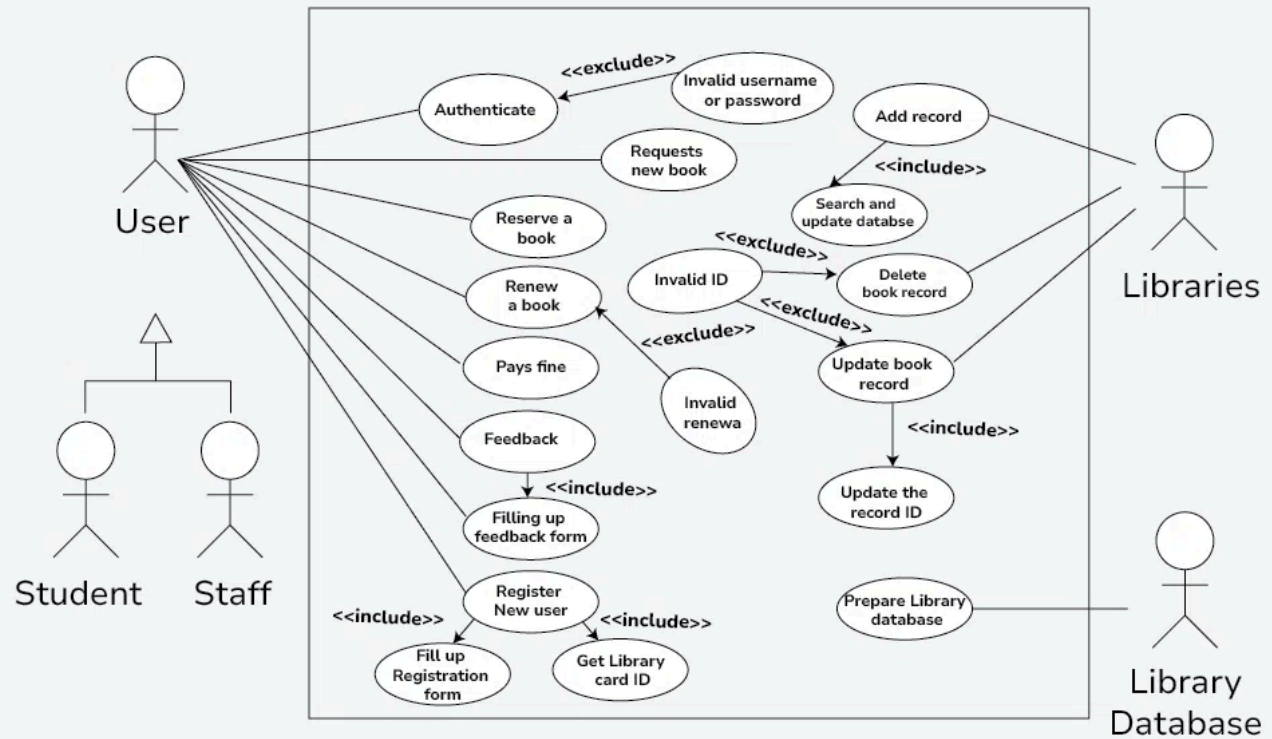


Figure - Sequence diagram of user login into a system

Use Case Diagram of Library Management System



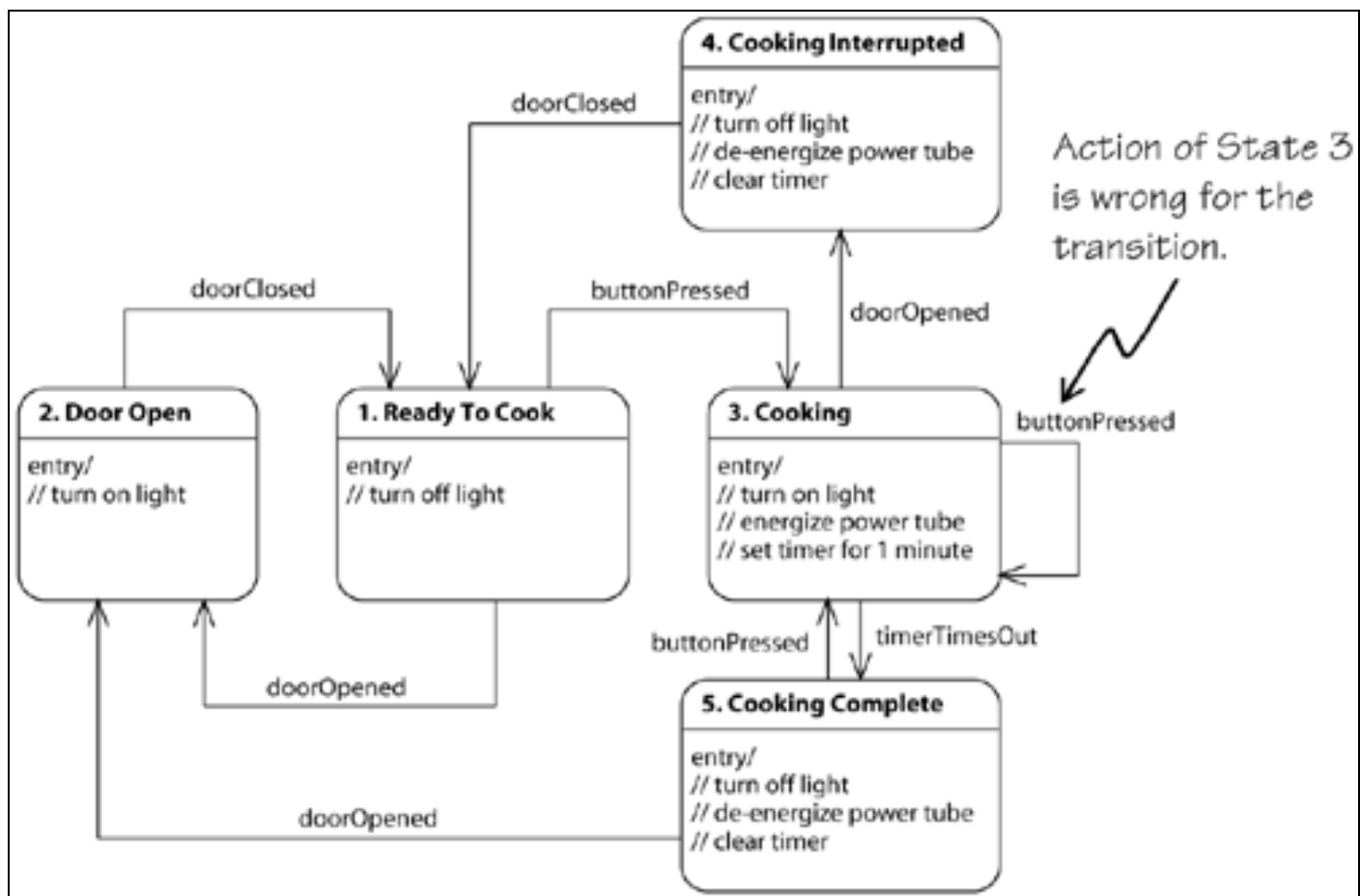
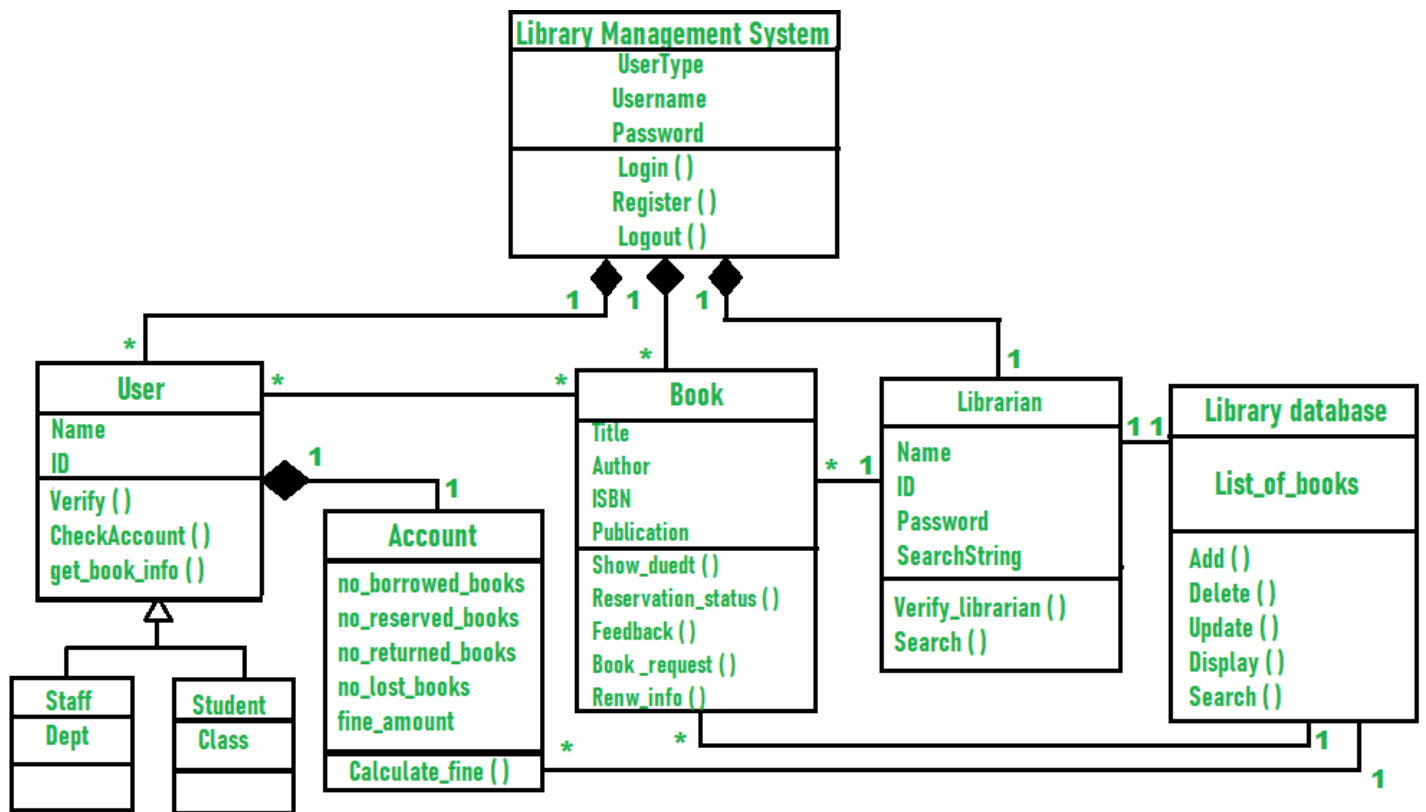


Figure - State diagram of a microwave oven



CLASS DIAGRAM FOR LIBRARY MANAGEMENT SYSTEM